Auditing Edge-IIoTset and Recovering Web-Attack Separability with Privacy-Preserving Protocol-Header Structural Features

Hamdan Ali

***Abstract*—Intrusion detection models evaluated on Edge-IIoTset routinely report accuracy above 99%, yet the benchmark contains build artifacts that a classifier can exploit without learning anything about network behavior. We audit the 157,800-row machine learning release and document four problems: placeholder zeros serialized inconsistently as `"0"` and `"0.0"` across the capture files from which the table was assembled, a comma fault in TShark exports that pushes text into a numeric column, eight constant fields, and 5,414 duplicate rows. We then remove the fifteen columns that identify the laboratory network, including addresses, timestamps, and raw payload strings. That step discards most of the evidence for web attacks, so before deleting the columns we convert them into 24 scalar descriptors of port roles, TCP options, payload entropy, and request structure. We call this representation privacy-preserving protocol-header structural feature engineering (PP-PSFE). On a stratified split of 152,386 flows, XGBoost trained on the resulting 62 features reaches 96.49% accuracy and 0.9618 macro-F1, ahead of logistic regression (84.87%) and random forest (94.38%) and level with LightGBM (96.33%), with a smaller train–test gap and 3.54 ms inference time per 1,000 flows. The five hardest HTTP-facing classes reach F1 between 0.86 and 0.93, against 0.44 and 0.66 for the linear baseline. We state plainly what this evidence does not establish, and we package the model as a REST service with a browser front end.**

***Index Terms*—Data leakage, Edge-IIoTset, explainable machine learning, gradient boosting, industrial Internet of Things, intrusion detection, TreeSHAP.**

I. Introduction

INDUSTRIAL networks used to be quiet places. A programmable logic controller talked to a few known peers over Modbus, and an air gap separated the plant floor from everything else. Much of that separation is gone. Sensors publish to MQTT brokers, edge gateways expose HTTP interfaces, and engineering workstations sit one routing hop from corporate IT [1], [2]. The attack surface grew with the connectivity, and intrusion detection systems (IDS) that learn from labeled traffic have become a common response.

Edge-IIoTset [3], [4] was built to support that work. It was captured on a multi-layer testbed with more than ten kinds of IoT and IIoT devices, and it labels benign traffic together with fourteen attack types. The machine learning release, `ML-EdgeIIoT-dataset.csv`, holds 157,800 rows and 63 columns. In the original study, 15-class accuracy ranged from 67.11% for a decision tree to 94.67% for a deep neural network [3]. Later papers report far higher figures, and the results are described as clustering above 99% [5].

Two things bothered us about that gap. A jump of five points or more from changing the classifier is unusual on a 15-class problem with heterogeneous protocols. And the wider network-IDS literature has repeatedly warned that benchmark accuracy says little about deployment, because datasets carry artifacts that models learn instead of attacks [6]–[11].

We came to the dataset to build a classifier, not to audit it. Loading the file with pandas raised a `DtypeWarning` on nine columns, and following that warning led to most of what this paper reports. The same zero appears as the string `"0"` in some capture files and as `"0.0"` in others, so a categorical encoder sees two categories where the network sees one absent field. Because normal and attack traffic were exported separately, the token tells a model which file a row came from. Galal [5] has described this artifact, and we cite that work as the primary reference for it. We reproduce the artifact on our own copy and add per-class counts for several fields, a second export fault, and an analysis of what remains after cleaning.

Cleaning has a cost. The columns that carry lab identity, namely addresses, timestamps, raw ports, and raw payload strings, also carry the content of web attacks. Deleting them leaves 38 protocol fields that say little about the difference between a SQL injection, a cross-site scripting probe, a file upload, a password guess, and an HTTP flood. Our response is to extract compact structural summaries from the dropped columns first, and to discard the originals afterwards.

This paper makes four contributions.

1) **A reproducible audit and sanitization procedure** for the ML release of Edge-IIoTset. It canonicalizes placeholder tokens, repairs a TShark column shift, removes eight constant fields and fifteen identifier or raw-string columns, and removes 5,414 duplicate rows (157,800 to 152,386 flows) before any split.
2) **PP-PSFE**, 24 scalar features computed from the columns that the sanitization step removes. They describe port roles, TCP option and payload structure, payload entropy, and application-layer lengths and lexical markers.
3) **A four-model benchmark** (logistic regression, random forest, LightGBM, XGBoost) with per-class results and TreeSHAP attributions on a stratified split in which every encoder and scaler is fitted on training data only.
4) **A deployable artifact**: the fitted preprocessing and model are bundled into one scikit-learn pipeline behind a

H. Ali is with the Department of Creative Technologies (Software Engineering), Faculty of Computing and Artificial Intelligence, Air University, Islamabad, Pakistan (e-mail: Hamdanaly651@gmail.com).

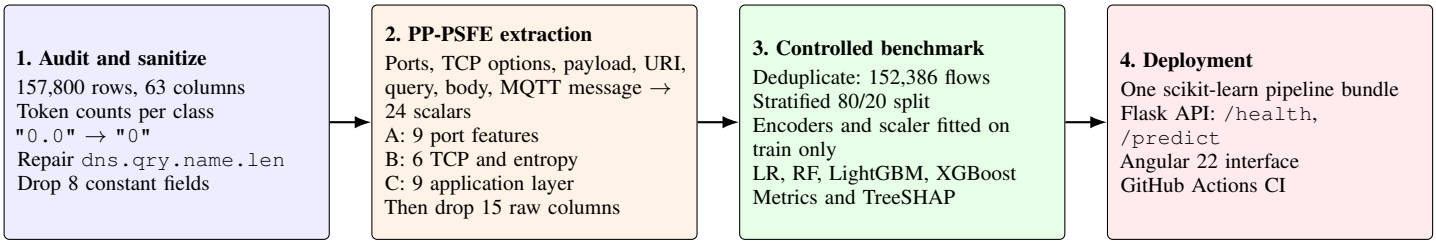


Fig. 1. Overview of the pipeline. Feature extraction happens before the identifying columns are removed, and nothing that is fitted ever sees the test split.

Flask service and an Angular interface, with continuous integration.

We also give a frank account of what the experiments cannot show (Section IX). In particular, all four models use the same 62 features, so the benefit of PP-PSFE is inferred from attributions and per-class behavior rather than from a feature ablation, and the evaluation uses a random split rather than a session-disjoint one.

The rest of the paper is organized as follows. Section II reviews related work, Section III presents the audit, Section IV describes sanitization and PP-PSFE, Section V gives the experimental setup, Sections VI and VII report results and explanations, Section VIII describes the service, and Sections IX and X discuss limits and conclude.

II. Related Work

Edge-IIoTset and reported accuracy. Ferrag *et al.* [3] released the dataset with centralized and federated baselines. Tareq *et al.* [12] analyzed it with deep models alongside two other IoT benchmarks. Since then it has become one of the standard testbeds for IIoT intrusion detection, and the headline accuracy figures have moved close to 100%.

Evaluation pitfalls in network IDS. Sommer and Paxson argued that anomaly and classification-based detection faces problems that do not arise in other machine learning applications [6], and Arp *et al.* catalogued recurring mistakes in security papers, including sampling bias and data snooping [7]. Kapoor and Narayanan show how leakage undermines reproducibility across machine-learning-based science more generally [13]. For network datasets in particular, Engelen *et al.* and Liu *et al.* documented construction and labeling errors in CIC-IDS-2017 and CSE-CIC-IDS-2018 [8], [9]. Flood *et al.* studied seven benchmark datasets and named six recurring design smells, among them features of outsized importance that have little to do with the attack mechanism [10]. Kostas *et al.* found that models built on individual packet features can reach near-perfect detection rates that fall by more than 90% in cross-session tests [11]. Roy examines shortcut learning and flow-level versus capture-disjoint splitting in IoT IDS evaluation [14].

The serialization artifact. Galal [5] shows that the standard preprocessing recipe for Edge-IIoTset turns a placeholder serialized differently in two branches of the dataset build into a near-perfect label proxy, and that file provenance becomes the label. That work goes on to rebuild a benchmark from raw captures. Our paper starts from the same observation on the same file but asks a different question: once the artifact and the identifying columns are gone, how much of the 15-class structure can be recovered from what the file still contains, and which of the recovered features can be trusted?

Models. We compare a linear baseline with three tree ensembles: random forest [15], LightGBM [16], and XGBoost [17]. Tree ensembles are a sensible choice for tabular data with mixed scales and many binary indicators [18], and they admit exact TreeSHAP attributions [19].

III. Dataset Audit

A. Procedure

We audited `ML-EdgeIIoT-dataset.csv` (157,800 rows, 63 columns, 15 classes). For every column we counted distinct string tokens by class, looked for values whose type disagreed with the rest of the column, checked variance, and compared rows after cleaning. The numbers below were measured on our copy of the file.

B. Findings

Table I summarizes five problems.

The direction of the token split depends on the column. In the four MQTT and DNS fields the benign class carries `"0"`, while in `http.request.method` the volumetric classes carry `"0.0"` and the web classes carry `"0"`. This is what one would expect if the table was assembled from separately exported capture files, each typed by its own content, though the file's history is not documented and we cannot confirm it.

We did not try to measure how much each problem inflates a model that is allowed to see it. Galal [5] does that with cross-validated measures. Our interest is in what a model can still learn after the problems are removed.

IV. Method

A. Canonical sanitization

Let v be a raw cell. For every categorical protocol field we apply

$$S(v) = \begin{cases} \texttt{"0"} & \text{if } \mathrm{trim}(v) \in \{\texttt{"0"}, \texttt{"0.0"}, \texttt{"0.00"}\}, \\ \mathrm{trim}(v) & \text{otherwise,} \end{cases} \tag{1}$$

so that one absent field has exactly one representation. For `dns.qry.name.len` we convert to a number and, where conversion fails because of the column shift, fall back to the character length of the offending string. We then drop the eight constant fields, the fifteen identifier and raw-string columns (after PP-PSFE extraction), and the second label column.

TABLE I
AUDIT OF THE ML RELEASE OF EDGE-IIOTSET (157,800 ROWS, 63 COLUMNS)

Problem	Fields	Evidence on our copy	Effect if untreated
1. Placeholder serialization ("0" vs. "0.0")	http.request.method, http.referer, http.request.version, mqtt.conack.flags, mqtt.protoname, mqtt.topic, dns.qry.name.len	In http.request.method, all 14,498 DDoS_UDP and 14,090 DDoS_ICMP rows hold "0.0"; 9,854 DDoS_HTTP and 9,751 XSS rows hold "0", and none hold "0.0". In dns.qry.name.len, mqtt.conack.flags, mqtt.protoname and mqtt.topic, Normal rows hold "0" (24,272; 23,012; 23,051; 23,055) and attack rows hold "0.0" (133,272; 133,499; 133,499; 133,499).	An encoder gives the two tokens different codes, so the token identifies the source file and therefore the class.
2. TShark column shift	dns.qry.name.len	93 rows carry a string in the numeric column: 91 hold _googlecast._tcp.local and 2 hold null. An unescaped comma inside a multicast DNS name moved later fields one column to the right.	Numeric conversion fails, or the column is treated as high-cardinality text.
3. Constant fields	icmp.unused, http.tls_port, dns.qry.type, dns.retransmit_request_in, mqtt.msg_decoded_as, mbtcp.len, mbtcp.trans_id, mbtcp.unit_id	Zero variance over all 157,800 rows.	No information; they dilute random feature subsampling.
4. Identity and session fields	15 columns: frame.time, ip.src_host, ip.dst_host, arp.src.proto_ipv4, arp.dst.proto_ipv4, tcp.srcport, tcp.dstport, udp.port, icmp.transmit_timestamp, tcp.options, tcp.payload, http.file_data, http.request.full_uri, http.request.uri.query, mqtt.msg	Attack scripts ran from fixed laboratory addresses, so addresses and timestamps are constant within a class.	A model can memorize the lab rather than the attack, and fails on any other subnet.
5. Duplicates	All 62 retained features	After cleaning, 5,414 rows (3.4%) are exact copies of other rows (157,800 to 152,386).	Identical rows can fall on both sides of a random split and inflate test scores.

B. PP-PSFE features

The premise is that a raw column mixes two things. One is topology-specific identity: addresses, times, random client ports, full URLs. The other is protocol structure: which role a port plays, how long a payload is, whether a TCP option is present, how random a byte sequence looks. The first must go. The second is what separates a flood from a login attempt. PP-PSFE keeps only scalar summaries of the second kind. We use "privacy-preserving" in a narrow sense: no address, hostname, timestamp, URL, or payload content survives in the final feature vector. We do not claim that the features carry no information about the testbed (Section IX).

Group A: port roles (9 features). Let p_s and p_d be the TCP source and destination ports; for non-TCP packets both are zero. We project them onto direction-invariant coordinates,

$$p_{\text{service}} = \begin{cases} \min(p_s, p_d) & \text{if } \min(p_s, p_d) < 10^4, \\ 0 & \text{otherwise,} \end{cases} \tag{2}$$

$$p_{\text{eph}} = \max(p_s, p_d), \qquad \Delta p = |p_s - p_d|, \tag{3}$$

$$d = \operatorname{sgn}(p_s - p_d), \tag{4}$$

and add five binary indicators: either port in $\{80, 443, 8080\}$ (psfe_is_http_service); either port equal to 1883 (psfe_is_mqtt_service); either port in $\{4321, 3333, 5900\}$ (psfe_is_iot_c2_service); either port equal to 65535 (psfe_is_boundary_probe); and a nonzero UDP port (psfe_udp_active).

Group B: TCP options and payload (6 features). From tcp.options we take the byte length, whether the field begins with 0204 (maximum segment size option), and whether it contains 080a (timestamp option). From tcp.payload we take the byte length, the value of the first byte, and the Shannon entropy [20] of the first 64 hexadecimal characters,

$$H = -\sum_{k=1}^{16} \hat{p}_k \log_2 \hat{p}_k, \qquad \hat{p}_k = \frac{n_k}{n}, \tag{5}$$

where n is the number of characters considered and n_k the count of hexadecimal symbol k. Because the alphabet has 16 symbols, $H \in [0, 4]$ bits per character; empty or single-character payloads score zero.

Group C: application layer (9 features). Before the URI, query, body and MQTT message columns are dropped, we record four lengths: the URI path length after the scheme and host are stripped, the query length, the HTTP body length (psfe_http_file_data_len), and the MQTT message byte length. We add five binary lexical markers over the lowercased URI, query and body, supplemented by short hexadecimal payload signatures: SQL injection tokens (select, union, extractvalue, sleep, %27, and similar), script injection tokens (script, alert, iframe), upload tokens (upload, multipart, hack.php), authentication tokens (login,

Algorithm 1 Leakage-controlled preprocessing and PP-PSFE

1: Load D_{raw} (157,800 rows)
2: $F \leftarrow$ PP-PSFE features computed from the raw port, TCP, URI, query, body and MQTT columns
3: Drop the 15 identifier/raw columns, 8 constant fields, and the second label
4: Repair `dns.qry.name.len`; apply $S(\cdot)$ of Eq. (1) to categorical fields
5: $D \leftarrow [D_{\text{core}} \,\|\, F]$; remove non-finite rows and duplicates (152,386 flows)
6: Stratified split of D into D_{train} (80%) and D_{test} (20%), seed 42
7: Fit encoders and scaler on D_{train} only; transform both splits
8: Fit each classifier on D_{train}; evaluate once on D_{test}

TABLE II
CLASSIFIER SETTINGS (150 TREES OR BOOSTING ROUNDS EACH)

Model	Settings
Logistic regression	L-BFGS, multinomial, $C = 1$, max 300 iterations
Random forest [15]	max depth 14, min. leaf 3, balanced class weights
LightGBM [16]	max depth 8, 63 leaves, learning rate 0.1, column subsample 0.8, balanced class weights[a]
XGBoost [17]	max depth 6, learning rate 0.1, row and column subsample 0.8, multiclass log-loss, default L2 penalty ($\lambda = 1$)

[a]Row subsampling in LightGBM requires `subsample_freq`> 0, which was not set.

`password`, `user_token`, `username`), and timeout or malformed-request status text.

C. Pipeline and split

Algorithm 1 gives the order of operations. Two choices matter. Deduplication happens before the split, so identical flows cannot straddle it. And the split happens before any encoder or scaler is fitted: categorical codes come from an ordinal encoder fitted on the training split with unseen categories mapped to -1, and a standard scaler is fitted on the training split only.

The final representation has $d = 62$ features: 32 numeric and 6 categorical core protocol fields, and the 24 PP-PSFE features.

V. EXPERIMENTAL SETUP

Data and split. After deduplication the corpus holds 152,386 flows. A stratified 80/20 split with seed 42 gives 121,908 training and 30,478 test flows. The test set is used once per model.

Models. Table II lists the settings. They are close to library defaults with moderate depth limits; we did not run a hyperparameter search, and we say so as a limitation.

Metrics. We report accuracy; weighted precision, recall and F1; macro-F1; one-vs-rest ROC-AUC (weighted); and macro-averaged PR-AUC. The overfitting gap is training accuracy minus test accuracy, and a gap above 5 points raises a warning in our code. Training time is wall-clock. Inference time is the time to predict the full test split, rescaled to 1,000 flows. All timings come from a single CPU-only Windows workstation running Python 3.12.10 with scikit-learn 1.6.0, XGBoost 3.2.0 and LightGBM 4.7.0, and should be read as relative rather than absolute.

VI. RESULTS

A. Overall performance

Table III and Fig. 2 report the four models on the held-out split. XGBoost has the highest test accuracy (96.49%) and macro-F1 (0.9618). LightGBM is close behind at 96.33% and 0.9596, and has the higher macro PR-AUC (0.9888 against 0.9874). The 0.16-point accuracy difference between the two is 49 flows out of 30,478. With one split and one seed we do not treat it as a ranking.

What separates them is cost and stability. XGBoost's train–test gap is 0.57 points against 2.10 for LightGBM, a factor of 3.7, and its inference time is 3.54 ms per 1,000 flows against 18.35 ms, a factor of 5.2 or roughly 282,000 flows per second on our machine. Random forest and logistic regression have the smallest gaps but give up 2.1 and 11.6 accuracy points respectively. The weakness of the linear model is not uniform, as the next subsection shows.

B. Per-class behavior

Table IV gives per-class scores. Eight classes are essentially solved by every tree model: `Backdoor`, `DDoS_TCP`, `DDoS_UDP`, `MITM`, `Normal`, `Port_Scanning`, `Ransomware`, and `DDoS_ICMP`. On the 4,831 benign test flows XGBoost has precision 0.9996 and recall 0.9998: one benign flow is flagged and two attack flows are labeled benign. `DDoS_TCP`, `Ransomware` and `MITM` reach F1 of 1.0000.

The interesting classes are the five that face HTTP. On the same 62 features, logistic regression scores F1 between 0.44 and 0.66 on them (`Password` 0.4371, `SQL_injection` 0.5746, `DDoS_HTTP` 0.6067, `Uploading` 0.6354, `XSS` 0.6585). XGBoost raises these to 0.8875, 0.9010, 0.8562, 0.9294 and 0.9285, gains of 45.0, 32.6, 25.0, 29.4 and 27.0 points. Fig. 3 shows the contrast across all classes, and Fig. 4 shows where the remaining mistakes fall.

Those mistakes are concentrated. Of the 1,069 test errors made by XGBoost, 975 (91.2%) are confusions between two of the five HTTP-facing classes, although these classes make up a third of the test set (10,122 of 30,478 flows). The largest single cells in Fig. 4 are `SQL_injection` predicted as `DDoS_HTTP` (203 flows), `Uploading` predicted as `XSS` (175), `Password` predicted as `DDoS_HTTP` (141), and `Password` predicted as `SQL_injection` (114). `DDoS_HTTP` remains the weakest class by F1 (0.8562). We read this as the visible limit of header-level and length-level evidence: a flood of valid requests and a short injection attempt can have similar sizes and flags.

VII. EXPLAINING THE MODEL

We computed exact TreeSHAP attributions [19] with XGBoost's native `pred_contribs` on held-out test flows, and

TABLE III
OVERALL PERFORMANCE ON THE HELD-OUT TEST SPLIT ($N_{\text{TEST}} = 30{,}478$)

Model	Train acc.	Test acc.	Prec. (wt)	Rec. (wt)	F1 (wt)	Macro-F1	ROC-AUC (wt)	PR-AUC (macro)	Gap	Train (s)	Infer. (ms/1k)
Logistic regression	0.8503	0.8487	0.8736	0.8487	0.8479	0.8419	0.9829	0.8656	+0.0016	23.18	0.43
Random forest	0.9458	0.9438	0.9455	0.9438	0.9438	0.9414	0.9978	0.9768	+0.0020	3.46	10.35
LightGBM	0.9843	0.9633	0.9640	0.9633	0.9632	0.9596	**0.9991**	**0.9888**	+0.0210	7.61	18.35
XGBoost	0.9706	**0.9649**	**0.9663**	**0.9649**	**0.9649**	**0.9618**	**0.9991**	0.9874	+0.0057	13.93	3.54

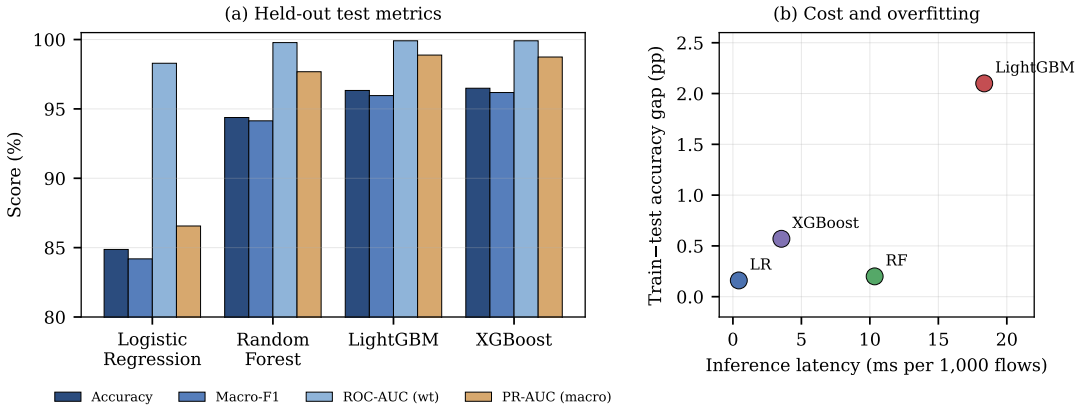


Fig. 2. (a) Test metrics for the four models. (b) Inference latency against the train–test accuracy gap. Values are those of Table III.

TABLE IV
PER-CLASS F1 FOR ALL MODELS AND PRECISION/RECALL FOR XGBOOST ($N_{\text{TEST}} = 30{,}478$)

Class	Support	LR F1	RF F1	LightGBM F1	XGB precision	XGB recall	XGB F1
`Backdoor`	1,973	0.9921	0.9997	0.9997	1.0000	0.9995	0.9997
`DDoS_HTTP`	2,112	0.6067	0.7780	0.8527	0.8304	0.8835	0.8562
`DDoS_ICMP`	2,619	0.9944	0.9996	0.9996	0.9996	0.9943	0.9969
`DDoS_TCP`	2,050	0.9634	1.0000	1.0000	1.0000	1.0000	1.0000
`DDoS_UDP`	2,900	1.0000	1.0000	0.9998	0.9997	1.0000	0.9998
`Fingerprinting`	171	0.9140	0.9855	0.9912	0.9086	0.9883	0.9468
`MITM`	78	1.0000	0.9750	0.9512	1.0000	1.0000	1.0000
`Normal`	4,831	1.0000	1.0000	0.9999	0.9996	0.9998	0.9997
`Password`	1,996	0.4371	0.8085	0.8806	0.9593	0.8257	0.8875
`Port_Scanning`	1,784	0.9606	0.9997	0.9997	1.0000	0.9994	0.9997
`Ransomware`	1,938	0.9918	1.0000	1.0000	1.0000	1.0000	1.0000
`SQL_injection`	2,057	0.5746	0.8392	0.8977	0.9013	0.9008	0.9010
`Uploading`	2,047	0.6354	0.8826	0.9214	0.9566	0.9038	0.9294
`Vulnerability_scanner`	2,012	0.9003	0.9749	0.9801	0.9845	0.9796	0.9821
`XSS`	1,910	0.6585	0.8775	0.9200	0.8718	0.9932	0.9285
Macro average	30,478	0.8419	0.9414	0.9596	0.9608	0.9645	0.9618
Weighted average	30,478	0.8479	0.9438	0.9632	0.9663	0.9649	0.9649

we also record the total loss reduction (gain) each feature earns across all trees. The 24 PP-PSFE features account for 73.88% of total gain and 57.1% of mean absolute SHAP magnitude, and they hold 12 of the top 20 SHAP positions (Fig. 5). Table V lists the twelve features with the largest gain.

The two rankings disagree in a useful way. Gain is dominated by binary indicators that make clean early splits: the MQTT port indicator alone takes 24.00% and the HTTP body length takes 19.20%. SHAP, which measures how far a feature moves the prediction, puts `psfe_service_port`, `tcp.flags` and `psfe_ephemeral_port` on top. Both views agree that the model leans mostly on the engineered features, and that the leaning is spread over many of them.

Attribution measures reliance and does not measure validity. Some of what the model relies on is protocol mechanics: TCP flag combinations separate scans and floods from ordinary sessions, and the body-length and injection-token features carry the information that distinguishes uploads and injections from floods. But the second- and third-ranked SHAP features involve the service and ephemeral port, and several core fields in the top 20 are counters or sequence values (`udp.stream`, `tcp.seq`, `tcp.ack`, `icmp.seq_le`). We return to this in Section IX.

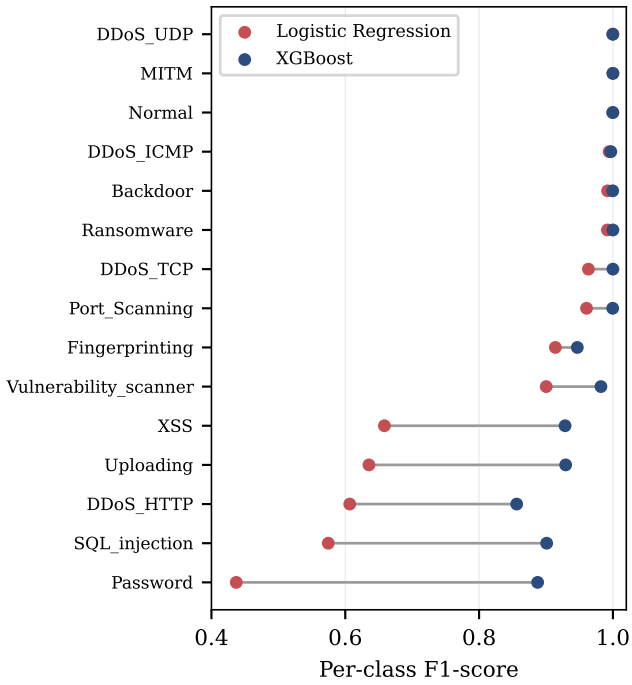


Fig. 3. Per-class F1 for logistic regression and XGBoost on the same 62 features, sorted by the linear baseline. The largest gaps are in the five HTTP-facing classes.

TABLE V
TWELVE FEATURES WITH THE LARGEST XGBOOST GAIN

#	Feature	Gain	Share	Origin
1	`psfe_is_mqtt_service`	1,800.27	24.00%	PP-PSFE
2	`psfe_http_file_data_len`	1,439.90	19.20%	PP-PSFE
3	`udp.stream`	642.45	8.57%	core
4	`psfe_tcp_opt_has_ts`	430.60	5.74%	PP-PSFE
5	`psfe_service_port`	334.18	4.46%	PP-PSFE
6	`psfe_is_iot_c2_service`	280.43	3.74%	PP-PSFE
7	`psfe_is_http_service`	250.37	3.34%	PP-PSFE
8	`http.content_length`	180.99	2.41%	core
9	`tcp.len`	167.36	2.23%	core
10	`tcp.flags`	167.05	2.23%	core
11	`psfe_app_sqli_flag`	144.06	1.92%	PP-PSFE
12	`tcp.connection.rst`	120.31	1.60%	core

VIII. DEPLOYMENT

Models that exist only in notebooks are hard to check and hard to reuse [21], so we packaged the work. The fitted `ColumnTransformer`, with string-safe wrappers around six ordinal encoders for the categorical fields and passthrough for the 56 numeric inputs in fixed column order, the scaler, and the XGBoost classifier are saved as one scikit-learn [22] pipeline (`model_bundle.joblib`). Keeping preprocessing and model in one object removes a common source of training–serving skew.

A Flask service exposes `GET /health` and `POST /predict`. An Angular 22 standalone interface calls it through a development proxy, offers one-click example flows (benign telemetry, `DDoS_TCP`, `SQL_injection`, `Ransomware`), and shows the predicted class with the five most probable classes (Fig. 6). Two GitHub Actions workflows run the backend `pytest` suite and the Angular production build, and publish the static interface to GitHub Pages. The live prediction endpoint runs locally; the Pages build disables prediction and says so.

The service takes the 62-dimensional feature vector as input. It does not yet extract PP-PSFE features from raw TShark output, so a client must compute them. Closing that gap is the most useful next engineering step. The interface is a research and teaching prototype, and predictions should be checked against other network telemetry before they drive automated blocking.

IX. LIMITATIONS AND THREATS TO VALIDITY

We list these in roughly the order a careful reviewer would raise them.

The benefit of PP-PSFE is not isolated. All four classifiers use the same 62 features. The large gap between the linear model and the boosted model on web classes mixes a change of model class with the presence of PP-PSFE, and neither the gain nor the SHAP shares prove that the engineered features are what improves the result. A direct test needs XGBoost trained on the 38 core features alone, and ablations by feature group. We provide a script for these runs with the code release but do not report them here.

The split is random. Flows from the same capture session fall on both sides of the split. Deduplication removes identical rows, not near-identical neighbors from one session. Results from session-disjoint or capture-disjoint evaluation, or from a second dataset, are the standard that Kostas *et al.* [11] and Roy [14] argue for, and we have not run them. "Leakage-controlled" in this paper therefore means that the specific artifacts in Section III and the fitting-order errors are removed, and no more.

Some PP-PSFE features can still encode the testbed. Equations (2)–(4) keep the service port and the larger port, which for client traffic is an ephemeral port that attack tools and operating systems pick in characteristic ways. The port sets in Group A were chosen by inspecting this corpus, and some hexadecimal signatures in Group C match paths of the vulnerable web application used as a target in the testbed (for example the fragment `GET /DVWA/vulnerabil`). Those features are not portable to another network. Their presence among the top SHAP features (Fig. 5) is a reason for caution about the 57.1% figure as evidence of general protocol understanding.

Retained core fields may also be identity-adjacent. Fields such as `udp.stream`, `tcp.seq`, `tcp.ack` and `icmp.seq_le` depend on capture order or sender stack. We kept them because they are standard header fields, but a stricter protocol would test their removal.

Statistical evidence is thin. Each model is trained once on one split with one seed, without confidence intervals. The XGBoost–LightGBM accuracy difference is within what we would expect from seed variation. Hyperparameters were not tuned, which may disadvantage the models differently.

Smaller points. Payload entropy is computed over hexadecimal characters of at most the first 64, rather than over

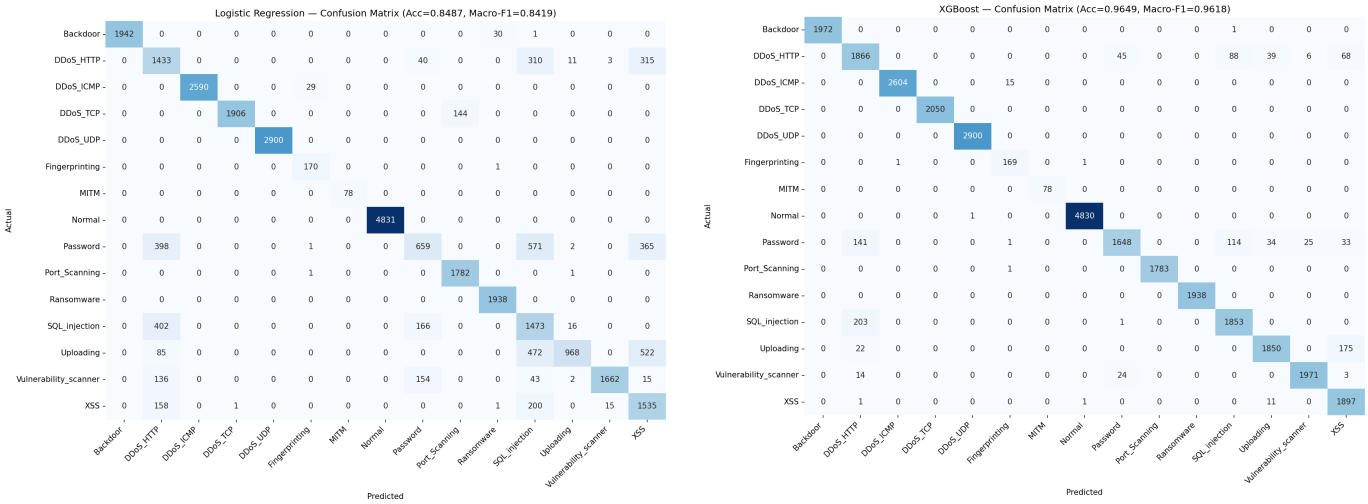


Fig. 4. Confusion matrices on the held-out split (rows: true class; columns: predicted class). Left: logistic regression. Right: XGBoost. The linear model spreads the five HTTP-facing classes across each other; XGBoost keeps most of that mass on the diagonal.

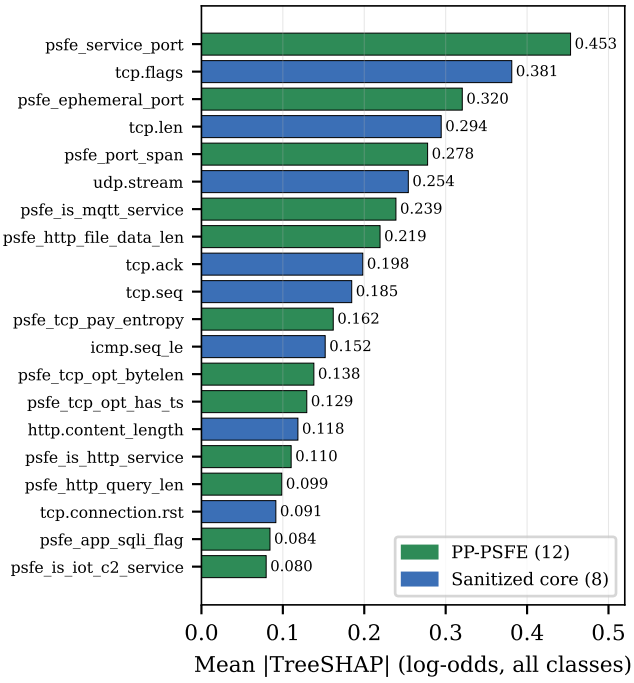


Fig. 5. Top 20 features by mean absolute TreeSHAP value for the XGBoost model. PP-PSFE features (green) hold 12 of the 20 positions.

decoded bytes. The TreeSHAP shares are computed on test flows and depend on the sample. Timings come from one CPU machine. The deployed service does not extract features from raw captures.

X. Conclusion

Edge-IIoTset's ML release contains a serialization artifact that makes file provenance readable as a class label, an export fault that corrupts a numeric column, constant columns, and thousands of duplicate rows. Removing them, together with the columns that identify the laboratory, is necessary and costly:

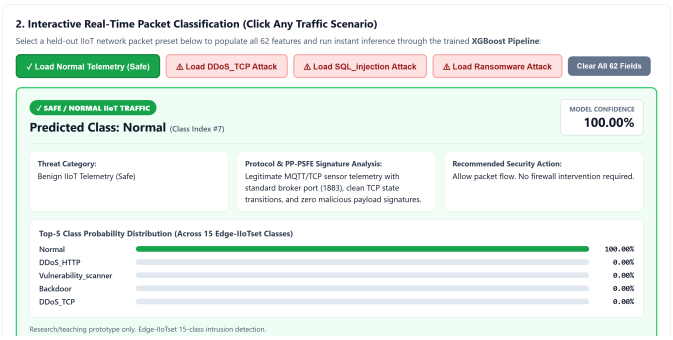


Fig. 6. Interactive classification panel of the Angular interface connected to the Flask service. A benign MQTT telemetry example is classified as `Normal` with the top-five class probabilities shown.

it discards most of what separates web attacks. Extracting 24 structural summaries before the removal gave a 62-feature representation on which XGBoost reached 96.49% accuracy and 0.9618 macro-F1 on 152,386 deduplicated flows, with most remaining errors confined to confusions among five HTTP-facing classes.

We would not call these numbers a generalization result. They come from a random split, a single seed, and features that may still carry some testbed regularities. The next experiments are a feature-group ablation, a session-disjoint or time-ordered evaluation, and a test on a second dataset; if PP-PSFE helps there, the argument for it becomes much stronger. Until then the audit procedure, the explicit feature definitions, and the packaged pipeline are the parts of this work that others can reuse directly.

Data and Code Availability

The Edge-IIoTset dataset is available from IEEE DataPort [4]. The training script, audit and sanitization code, service, interface, tests and workflows are in the public repository https://github.com/Hamdanali3/Edge-IIoTset_From-a-Trained-ML-Model-to-a-Full-Stack-Research-Application.

REFERENCES

[1] E. Sisinni, A. Saifullah, S. Han, U. Jennehag, and M. Gidlund, "Industrial Internet of Things: Challenges, opportunities, and directions," *IEEE Trans. Ind. Informat.*, vol. 14, no. 11, pp. 4724–4734, Nov. 2018.

[2] M. Zolanvari, M. A. Teixeira, L. Gupta, K. M. Khan, and R. Jain, "Machine learning-based network vulnerability analysis of Industrial Internet of Things," *IEEE Internet Things J.*, vol. 6, no. 4, pp. 6822–6834, Aug. 2019.

[3] M. A. Ferrag, O. Friha, D. Hamouda, L. Maglaras, and H. Janicke, "Edge-IIoTset: A new comprehensive realistic cyber security dataset of IoT and IIoT applications for centralized and federated learning," *IEEE Access*, vol. 10, pp. 40 281–40 306, 2022.

[4] ——, "Edge-IIoTset: A new comprehensive realistic cyber security dataset of IoT and IIoT applications," IEEE DataPort, 2022.

[5] M. M. Galal, "Provenance, not behaviour: A serialisation artifact in Edge-IIoTset and a leakage-free benchmark for precision-agriculture intrusion detection," arXiv:2608.15761, Aug. 2026.

[6] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *Proc. IEEE Symp. Security Privacy*, Oakland, CA, USA, 2010, pp. 305–316.

[7] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck, "Dos and don'ts of machine learning in computer security," in *Proc. 31st USENIX Security Symp.*, Boston, MA, USA, 2022, pp. 3971–3988.

[8] G. Engelen, V. Rimmer, and W. Joosen, "Troubleshooting an intrusion detection dataset: The CICIDS2017 case study," in *Proc. IEEE Security Privacy Workshops (SPW)*, 2021, pp. 7–12.

[9] L. Liu, G. Engelen, T. Lynar, D. Essam, and W. Joosen, "Error prevalence in NIDS datasets: A case study on CIC-IDS-2017 and CSE-CIC-IDS-2018," in *Proc. IEEE Conf. Commun. Netw. Secur. (CNS)*, 2022, pp. 254–262.

[10] R. Flood, G. Engelen, D. Aspinall, and L. Desmet, "Bad design smells in benchmark NIDS datasets," in *Proc. IEEE 9th Eur. Symp. Security Privacy (EuroS&P)*, Vienna, Austria, Jul. 2024, pp. 658–675.

[11] K. Kostas, M. Just, and M. A. Lones, "Individual packet features are a risk to model generalization in ML-based intrusion detection," *IEEE Netw. Lett.*, vol. 7, no. 1, pp. 66–70, Mar. 2025.

[12] I. Tareq, B. M. Elbagoury, S. El-Regaily, and E.-S. M. El-Horbaty, "Analysis of TON-IoT, UNW-NB15, and Edge-IIoT datasets using DL in cybersecurity for IoT," *Appl. Sci.*, vol. 12, no. 19, p. 9572, Sep. 2022.

[13] S. Kapoor and A. Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science," *Patterns*, vol. 4, no. 9, p. 100804, 2023.

[14] U. S. Roy, "Unmasking shortcut learning in IoT intrusion detection: A forensic, multi-paradigm evaluation of feature dependence and data leakage," arXiv:2609.28725, 2026.

[15] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, Oct. 2001.

[16] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 30, 2017, pp. 3146–3154.

[17] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Aug. 2016, pp. 785–794.

[18] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why do tree-based models still outperform deep learning on typical tabular data?" in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS) Datasets Benchmarks Track*, vol. 35, 2022.

[19] S. M. Lundberg, G. Erion, H. Chen, A. DeGrave, J. M. Prutkin, B. Nair, R. Katz, J. Himmelfarb, N. Bansal, and S.-I. Lee, "From local explanations to global understanding with explainable AI for trees," *Nature Mach. Intell.*, vol. 2, no. 1, pp. 56–67, Jan. 2020.

[20] C. E. Shannon, "A mathematical theory of communication," *Bell Syst. Tech. J.*, vol. 27, no. 3, pp. 379–423, Jul. 1948.

[21] D. Sculley *et al.*, "Hidden technical debt in machine learning systems," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 28, 2015, pp. 2503–2511.

[22] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.